1. Importazione delle librerie

python

from scapy.all import sniff, IP, TCP, UDP

Questa riga importa quattro elementi dalla libreria Scapy.

- sniff: cattura i pacchetti che transitano su un'interfaccia di rete.
- IP: permette di riconoscere e leggere il livello IP di un pacchetto.
- TCP: permette di riconoscere e leggere i dati del protocollo TCP.
- UDP: permette di riconoscere e leggere i dati del protocollo UDP.

Per usare questo programma, Scapy deve essere installato. Puoi farlo con:

bash

pip install scapy

Su alcuni sistemi operativi, la cattura dei pacchetti richiede privilegi amministrativi.

2. Definizione della funzione di analisi

python

def analizza_pacchetto(pacchetto):

- def serve a definire una funzione.
- analizza_pacchetto è il nome scelto per la funzione.
- pacchetto è il parametro che riceverà il pacchetto da analizzare.

La funzione è progettata per ricevere un pacchetto catturato e mostrarne alcune informazioni.

Importante: definire una funzione non significa eseguirla. Per eseguirla, bisogna chiamarla esplicitamente o passarla a un'altra funzione che la richiami.

3. Verifica della presenza del livello IP

python

if IP in pacchetto:

Questa condizione verifica se il pacchetto contiene un livello IP riconosciuto da Scapy.

Se è presente, il programma esegue le istruzioni successive.

Se non è presente, il blocco non viene eseguito e la funzione termina senza stampare informazioni.

4. Lettura degli indirizzi e del protocollo IP

python

sorgente = pacchetto[IP].src

Legge l'indirizzo IP sorgente del pacchetto e lo salva nella variabile sorgente.

È l'indirizzo da cui il pacchetto risulta provenire.

python

destinazione = pacchetto[IP].dst

Legge l'indirizzo IP di destinazione e lo salva nella variabile destinazione.

È l'indirizzo IP a cui il pacchetto è diretto.

python

protocollo = pacchetto[IP].proto

Legge il numero che identifica il protocollo contenuto nel pacchetto IP.

Alcuni valori comuni sono:

- 6: TCP.
- 17: UDP.
- 1: ICMP.

Questa variabile contiene un numero identificativo, non necessariamente il nome testuale del protocollo.

5. Stampa delle informazioni IP

python

print("-" * 60)

Stampa 60 trattini per separare visivamente le informazioni relative ai pacchetti.

python

print(f"IP sorgente: {sorgente}")

Mostra l'indirizzo IP sorgente.

La f davanti alla stringa permette di inserire il valore della variabile tramite {sorgente}.

python

print(f"IP destinazione: {destinazione}")

Mostra l'indirizzo IP di destinazione.

Per esempio, l'output potrebbe essere:

text

IP sorgente: 192.168.1.20

IP destinazione: 142.250.184.14

Gli indirizzi sono puramente esemplificativi.

6. Controllo del protocollo TCP

python

if TCP in pacchetto:

Verifica se Scapy riconosce nel pacchetto un livello TCP.

Se la condizione è vera, vengono eseguite le istruzioni seguenti:

python

print("Protocollo: TCP")

Stampa il nome del protocollo.

python

print(f"Porta sorgente: {pacchetto[TCP].sport}")

Legge e stampa la porta TCP sorgente.

pacchetto[TCP] accede al livello TCP del pacchetto, mentre .sport restituisce la porta sorgente.

python

print(f"Porta destinazione: {pacchetto[TCP].dport}")

Legge e stampa la porta TCP di destinazione, attraverso .dport.

Per esempio:

text

Protocollo: TCP

Porta sorgente: 51524

Porta destinazione: 443

In questo esempio, il pacchetto usa la porta sorgente 51524 e la porta di destinazione 443, comunemente utilizzata per HTTPS.

7. Controllo del protocollo UDP

python

elif UDP in pacchetto:

elif significa “altrimenti, se”.

Il controllo UDP viene eseguito soltanto se la condizione TCP precedente è falsa.

Se il pacchetto contiene UDP, il programma esegue:

python

print("Protocollo: UDP")

Stampa il nome del protocollo UDP.

python

print(f"Porta sorgente: {pacchetto[UDP].sport}")

Legge la porta UDP sorgente.

python

print(f"Porta destinazione: {pacchetto[UDP].dport}")

Legge la porta UDP di destinazione.

Queste informazioni sono analoghe a quelle mostrate per TCP, ma vengono estratte dal livello UDP.

8. Gestione degli altri protocolli IP

python

else:

  print(f"Protocollo IP: {protocollo}")

Questo else viene eseguito quando il pacchetto contiene IP, ma non viene riconosciuto come TCP o UDP.

Il programma stampa il numero identificativo del protocollo IP.

Per esempio, se il pacchetto è ICMP, potrebbe mostrare:

text

Protocollo IP: 1

Questa condizione permette di gestire anche pacchetti IP che non appartengono ai due protocolli TCP e UDP.

9. Messaggi iniziali

python

print("Cattura del traffico avviata...")

Mostra un messaggio per indicare che sta per iniziare la cattura.

python

print("Premi CTRL+C per interrompere.")

Comunica all'utente come interrompere manualmente il programma.

La combinazione CTRL+C genera normalmente un'interruzione da tastiera.

10. Avvio della cattura con sniff()

python

try:

  sniff(

    filter="tcp port 80 or tcp port 443",

      prn=lambda p: p.summary(),

      store=False

)

Questa è la parte che avvia concretamente la cattura dei pacchetti.

try

python

try:

Avvia un blocco in cui il programma può gestire l'interruzione tramite except.

sniff()

python

sniff(

Chiama la funzione Scapy che ascolta il traffico di rete e riceve i pacchetti che soddisfano i filtri impostati.

filter

python

filter="tcp port 80 or tcp port 443",

Definisce un filtro per selezionare soltanto i pacchetti TCP che utilizzano la porta 80 oppure la porta 443, come porta sorgente o destinazione.

- tcp port 80: traffico TCP associato alla porta 80, comunemente usata da HTTP.
- or: significa “oppure”.

- tcp port 443: traffico TCP associato alla porta 443, comunemente usata da HTTPS.

Attenzione: questo filtro non cattura tutto il traffico TCP e non cattura i pacchetti UDP, anche se la funzione analizza_pacchetto() contiene un controllo per UDP.

prn

python

prn=lambda p: p.summary(),

Questo parametro specifica quale operazione eseguire per ogni pacchetto catturato.

- prn riceve una funzione da eseguire per ciascun pacchetto.
- lambda p: definisce una piccola funzione anonima che riceve il pacchetto nella variabile p.
- p.summary() restituisce una breve descrizione del pacchetto.

Per esempio, potresti vedere una riga simile a:

text

IP 192.168.1.20.51524 > 142.250.184.14.443: Flags [S], ...

Il formato esatto dipende dal pacchetto.

Questa è la differenza fondamentale del tuo codice: prn=lambda p: p.summary() non chiama analizza_pacchetto(p). Di conseguenza, la funzione che hai definito all'inizio non viene utilizzata dalla cattura.

store=False

python

store=False

Indica a Scapy di non conservare in memoria tutti i pacchetti catturati.

I pacchetti vengono elaborati man mano che arrivano, evitando di accumulare l'intera cattura in una lista.

È utile quando si vuole osservare il traffico per un periodo prolungato.

11. Interruzione della cattura

python

```
except KeyboardInterrupt:
    print("\nCattura terminata.")
```

Se l'utente interrompe il programma con CTRL+C, Python genera normalmente un'eccezione KeyboardInterrupt.

- except KeyboardInterrupt: intercetta tale interruzione.
- print("\nCattura terminata.") stampa un messaggio finale.
- \n inserisce una nuova riga prima del messaggio.

Questo permette di terminare la cattura in modo più ordinato.

Riassunto del funzionamento

1. Importa Scapy

2. Definisce la funzione di analisi

3. Avvia la cattura TCP sulle porte 80 e 443

4. Stampa una sintesi di ogni pacchetto

Nel codice originale viene eseguito p.summary(), non analizza_pacchetto().

5. Si interrompe con CTRL+C